

GUIA DE ADMINISTRAÇÃO

SSH e Linux na VM

Como conectar na VM do agente, navegar pelos arquivos, editar configuração e rodar os comandos do dia a dia — PM2, Nginx, logs, memória e git.

[Conexão SSH](#)[Comandos essenciais](#)[Rotina do projeto](#)[Troubleshooting](#)

Repositório: github.com/andre-rizzato/ai-customer-service-agent

VM: vm-agente — rg-agente-atendimento (Azure, eastus)

Versão do documento: 1.0 — Outubro/2026

Sumário

1. Conectando via SSH
2. Navegação e arquivos
3. Ver e editar conteúdo
4. Processos com PM2
5. Logs
6. Sistema — memória, disco, serviços
7. Rede e testes
8. Nginx e certificado TLS
9. Git
10. Receitas prontas (rotina real do projeto)
11. Segurança e boas práticas

1. Conectando via SSH

SSH (*Secure Shell*) é como você abre um terminal dentro da VM, de dentro do seu próprio terminal. O comando usado neste projeto:

```
ssh -i ~/.ssh/id_rsa azureuser@20.127.12.103
```

Parte	O que é
<code>-i</code> <code>~/.ssh/id_rsa</code>	Caminho da sua chave privada — a "senha" criptográfica que autentica você na VM, sem precisar digitar senha nenhuma
<code>azureuser</code>	Usuário do sistema operacional dentro da VM
<code>20.127.12.103</code>	IP público da VM (<code>vm-agente</code>)

Primeira conexão

Na primeira vez, o terminal pergunta se confia no "host" (a VM) — responda `yes`. Isso fica salvo em `~/.ssh/known_hosts` e não pergunta de novo.

A chave privada nunca sai da sua máquina

`~/.ssh/id_rsa` é o que garante que só você entra na VM. Não copie, não cole em chat nenhum, não mande por email.

Pra sair da VM e voltar pro seu terminal normal: `exit` ou `Ctrl+D`.

2. Navegação e arquivos

Comando	O que faz
<code>pwd</code>	Mostra em qual pasta você está agora
<code>ls</code> / <code>ls -la</code>	Lista arquivos da pasta atual (<code>-la</code> : inclui ocultos, com detalhes — dono, tamanho, data)
<code>cd nome-da-pasta</code>	Entra numa pasta. <code>cd ..</code> sobe um nível, <code>cd ~</code> vai pra sua pasta pessoal (<code>/home/azureuser</code>)
<code>mkdir nome</code>	Cria uma pasta nova
<code>cp origem destino</code>	Copia um arquivo. <code>cp -r</code> copia pasta inteira
<code>mv origem destino</code>	Move ou renomeia (não existe comando "renomear" separado — <code>mv</code> faz os dois)
<code>rm arquivo</code>	Apaga um arquivo. <code>rm -r pasta</code> apaga pasta inteira — sem lixeira, é definitivo
<code>find . -name "*.log"</code>	Procura arquivos por nome, a partir da pasta atual

Cuidado com `rm -rf`

Apaga sem pedir confirmação e sem lixeira. Antes de rodar, confirme com `pwd` e `ls` que está na pasta certa.

As duas pastas que importam neste projeto:

```
~/agente-atendimento/ # o agente Node (RAG + LLM + canais)
~/agent-service/      # o AgentService Python (DistributedOrderSystem)
```

3. Ver e editar conteúdo

Comando	O que faz
<code>cat arquivo</code>	Mostra o conteúdo inteiro de uma vez — bom pra arquivos curtos (<code>.env</code> , <code>agent.config.json</code>)
<code>less arquivo</code>	Mostra um arquivo grande por partes, navegável (setas, <code>q</code> pra sair)
<code>tail -n 20 arquivo</code>	Mostra só as últimas 20 linhas
<code>tail -f arquivo</code>	Mostra as últimas linhas e continua mostrando em tempo real — ótimo pra acompanhar log ao vivo
<code>nano arquivo</code>	Abre um editor de texto simples direto no terminal — <code>Ctrl+O</code> salva, <code>Ctrl+X</code> sai
<code>grep "termo" arquivo</code>	Procura um texto dentro de um arquivo. <code>grep -r "termo" pasta/</code> procura em todos os arquivos de uma pasta

Editar `.env` ou `agent.config.json`

`nano ~/agente-atendimento/.env` → edita → `Ctrl+O` , Enter, `Ctrl+X` . Depois de editar, sempre reinicie o processo (Seção 4) pra aplicar.

4. Processos com PM2

PM2 é quem mantém o agente Node e o AgentService rodando continuamente, reinicia sozinho se travar, e sobrevive a reboot da VM.

Comando	O que faz
<code>pm2 list</code>	Mostra todos os processos — status (online/stopped), uso de CPU/memória, quantas vezes reiniciou
<code>pm2 logs nome</code>	Mostra os logs ao vivo de um processo (<code>Ctrl+C</code> pra sair)
<code>pm2 logs nome --lines 50 --nostream</code>	Mostra as últimas 50 linhas e volta pro terminal (não fica "preso" acompanhando)
<code>pm2 restart nome</code>	Reinicia um processo — necessário depois de editar <code>.env</code> ou <code>agent.config.json</code>
<code>pm2 restart nome --update-env</code>	Igual acima, mas também recarrega variáveis de ambiente novas (ex.: depois de adicionar um secret no Key Vault)
<code>pm2 stop nome</code> / <code>pm2 start nome</code>	Para e liga um processo, sem removê-lo da lista
<code>pm2 save</code>	Importante depois de qualquer mudança — grava o estado atual pra sobreviver a um reboot da VM

Os dois processos deste projeto: `agente-atendimento` e `agent-service`.

5. Logs

Onde	Comando
Logs da aplicação (saída normal)	<code>pm2 logs nome --lines 30 --nostream</code>
Só os erros	<code>pm2 logs nome --lines 30 --nostream --err</code>
Requisições que chegaram no Nginx	<code>sudo tail -30 /var/log/nginx/access.log</code>
Erros do Nginx (ex.: proxy falhando)	<code>sudo tail -30 /var/log/nginx/error.log</code>
Log de um serviço systemd (ex.: o que sobe o PM2 no boot)	<code>journalctl -u pm2-azureuser -n 30</code>

6. Sistema — memória, disco, serviços

Comando	O que faz
<code>free -h</code>	Memória RAM usada/livre (<code>-h</code> : formato legível, MB/GB). A VM é pequena (892MB) — confira antes de instalar algo pesado
<code>df -h</code>	Espaço em disco usado/livre
<code>uptime</code>	Há quanto tempo a VM está ligada, e carga média de CPU
<code>top</code>	Lista processos ao vivo por uso de CPU/memória (<code>q</code> pra sair)
<code>systemctl status nome</code>	Status de um serviço do sistema (ex.: <code>nginx</code> , <code>pm2-azureuser</code>)
<code>sudo systemctl restart nome</code>	Reinicia um serviço do sistema

VM pequena — cuidado com instalação/build

Já tivemos um incidente de memória esgotada rodando `npm ci` com os processos ativos ao mesmo tempo. Antes de instalar algo pesado, considere `pm2 stop` nos dois processos, e confira `free -h` antes e depois.

7. Rede e testes

Comando	O que faz
<code>curl -s URL</code>	Faz uma requisição HTTP e mostra a resposta — útil pra testar se o servidor está respondendo
<code>curl -s -o /dev/null -w "% {http_code}\n" URL</code>	Mostra só o código de status HTTP (200, 404, 500...), sem o corpo da resposta
<code>curl -I URL</code>	Mostra só os headers da resposta
<code>ss -tlnp</code>	Lista quais portas estão escutando na VM (equivalente moderno do antigo <code>netstat</code>)
<code>ping -c 4 host</code>	Testa se um host responde na rede (4 tentativas)

Pra conferir se o agente está de pé, de dentro da própria VM:

```
curl -s http://localhost:3000/health
curl -s http://localhost:8100/health
```

8. Nginx e certificado TLS

Comando	O que faz
<code>sudo nginx -t</code>	Testa se a configuração do Nginx está válida, antes de recarregar — evita derrubar o site com um erro de sintaxe
<code>sudo systemctl reload nginx</code>	Aplica mudanças de configuração sem derrubar conexões ativas
<code>sudo certbot certificates</code>	Lista os certificados TLS instalados e quando expiram
<code>sudo certbot renew --dry-run</code>	Testa a renovação automática sem renovar de verdade (a renovação real já roda sozinha agendada)

9. Git

Comando	O que faz
<code>git status</code>	Mostra o que mudou localmente, comparado ao último commit
<code>git pull</code>	Baixa e aplica os commits mais recentes do GitHub
<code>git log --oneline -10</code>	Mostra os últimos 10 commits, resumidos

Normalmente você não precisa disso na VM

O deploy é automático via GitHub Actions a cada push — a VM se atualiza sozinha. `git pull` manual só é necessário se estiver investigando algo ou testando antes do CI rodar.

10. Receitas prontas — rotina real do projeto

Editei o `.env` ou `agent.config.json`, como aplico?

```
pm2 restart agente-atendimento --update-env
pm2 save
```

Adicionei um secret novo no Key Vault, como o processo pega ele?

```
pm2 restart agente-atendimento --update-env
# (o processo busca os secrets do Key Vault de novo, no startup)
```

O site/bot parece fora do ar — o que eu confiro, em ordem?

- 1 `pm2 list` — os dois processos estão `online`?
- 2 `curl -s http://localhost:3000/health` — responde `{"ok":true}`?
- 3 `pm2 logs agente-atendimento --lines 30 --nostream --err` — tem erro recente?
- 4 `free -h` — a VM não está sem memória?

A VM ficou lenta ou travou depois de instalar algo

```
free -h # confirma se é falta de memória
pm2 stop agente-atendimento agent-service # libera memória
# ... termina a instalação/build ...
pm2 restart agente-atendimento agent-service --update-env
pm2 save
```

Atualizar o código manualmente (fora do CI/CD automático)

```
cd ~/agente-atendimento
git pull
npm ci
npm run build
pm2 restart agente-atendimento --update-env
pm2 save
```

Conferir se o certificado HTTPS está válido

```
sudo certbot certificates
curl -sI https://rizzato-tech.rizzatotech.com | head -1
```

11. Segurança e boas práticas

- ☐ Nunca cole senha, token ou chave de API direto no terminal de um jeito que fique salvo no histórico sem necessidade — prefira variável de ambiente ou o Key Vault
- ☐ `sudo` só quando o comando realmente precisar — não vire hábito de colocar na frente de tudo
- ☐ Antes de qualquer `rm -r`, confirme com `pwd` e `ls` que está na pasta certa
- ☐ A chave privada (`~/.ssh/id_rsa`) é pessoal — nunca envie por chat, email ou deixe em repositório
- ☐ Depois de qualquer mudança que deva sobreviver a um reboot da VM, rode `pm2 save`